Improving the RGNN Pipeline: Learned Pooling and Learned Graph Construction

Research proposals following Experiment 08

July 16, 2026

The current RGNN pipeline

Experiment 08 established the following pipeline as the best performing formulation. Nodes (e.g. pregnancy visits) carry 64-dimensional MiniLM feature vectors, and each node belongs to exactly one *color* (e.g. a patient). The graph contains two edge types: (1) a kNN graph (k=20) over the MiniLM node features, and (2) *same-color* edges connecting every pair of nodes that share a color. A relational GNN — RGAT, with RGCN as a close second — processes both relations jointly, using a separate learned transformation (and, for RGAT, separate attention) per relation, and produces an embedding per node. Two heads read these embeddings: a *node head* classifies each node directly from its embedding, and a *color head* classifies each color by **mean-pooling** the embeddings of all its member nodes and passing the pooled vector through an MLP. Both heads are trained jointly ($\mathcal{L} = \mathcal{L}_{\text{node}} + \mathcal{L}_{\text{color}}$). On `pregnancy_MiniLM_L6` this reaches $.818 \pm .009$ test AUC on the node task and $.798 \pm .006$ on the color task — roughly $+20$ AUC points over every non-relational variant.

Two components of this pipeline remain *unlearned*: the mean-pooling that turns node embeddings into a color representation is parameter-free and treats every node as equally important, and the kNN graph is fixed before training ever sees a label. These are exactly the two channels the proposals below address.

Channel 1: Pooling (node embeddings → color prediction)

1.1 Attention (MIL) pooling. The color task is an instance of *Multiple Instance Learning*: a color is a "bag" of nodes, and the bag label must be predicted from its instances. Gated attention pooling (Ilse et al., 2018) replaces the mean with a learned weighted average: a small MLP scores every node, the scores are softmax-normalized *within each color*, and the color representation is the weighted sum $\sum_i \alpha_i h_i$. Nodes that carry the signal are amplified; uninformative episodes are suppressed. It is a drop-in replacement for the current `scatter_mean_by_color`, adds negligible cost, and the attention weights α_i are directly interpretable — one can inspect *which* visits drove each patient-level prediction.

1.2 Color supernodes as a third relation. The most RGNN-native option: add one *virtual node* per color, connected to all of the color's member nodes by a new, third edge type. The relational GNN then learns the aggregation itself through message passing — multi-hop and (with RGAT) attention-weighted — and the supernode's final embedding feeds the color head directly, eliminating the external pooling step altogether. This unifies pooling and representation learning into a single mechanism, and additionally lets color-level information propagate *back down* to the member nodes, which may also help the node task. Given that adding the same-color relation was the single largest win in Experiment 08, extending the relational vocabulary further is a natural continuation.

1.3 Multi-statistic readout. A minimal upgrade that should be run regardless, as a baseline for 1.1 and 1.2: concatenate mean ‖ max ‖ sum pooling before the color MLP. The mean alone washes out "one abnormal node" patterns that max-pooling preserves, while the sum retains bag-size information that both mean and max discard.

1.4 Set-transformer readout (long shot). A learned query vector attends over the color's node set with multi-head attention (Pooling by Multihead Attention, Lee et al., 2019), optionally preceded by self-attention blocks among the member nodes. Strictly more expressive than 1.1 — it can model interactions *between* member nodes at readout time. I believe it's worth attempting only if 1.1 shows gains and colors contain enough nodes to exploit the extra capacity (currently, they do not).

Channel 2: Graph construction

2.1 Multiple graphs as multiple relations. The pipeline already pays for a relational GNN but feeds it only two relations. Rather than committing to a single kNN graph, supply several complementary edge types and let the per-relation weights (and RGAT's per-relation attention) decide what matters: kNN at a small k (tight neighborhoods) alongside kNN at a large k (broader context); cosine-kNN versus Euclidean-kNN. This directly generalizes the design that already produced the main win.

2.2 Mutual kNN and similarity-weighted edges. Plain kNN is asymmetric and noisy in dense regions: hub nodes accumulate many low-quality incoming edges. Two cheap filters: (a) *mutual* kNN, keeping an edge only if each endpoint is among the other's k nearest neighbors, and (b) pruning edges whose feature similarity falls below a threshold, or weighting messages by that similarity.

2.3 Dynamic graph rebuilding. Start from the feature-space kNN graph, then every N epochs rebuild the kNN relation *in the current embedding space* of the GNN (in the style of DGCNN's dynamic graphs). The topology thus adapts to what the model has learned — nodes that the task considers similar become neighbors even if their raw MiniLM features are distant. The main tuning knobs are the rebuild frequency and whether to keep the original graph as an additional (static) relation, which combines naturally with 2.1.

2.4 Fully learned graph. The ambitious endpoint: learn the topology end-to-end with the classifier, e.g. a Differentiable Graph Module that learns a metric under which kNN is computed inside the forward pass, or Gumbel-top-k sampling of edges from a learned score matrix. This has the highest ceiling and the greatest novelty value, but is markedly harder to tune and slower to train.